MERN Stack Ticketing App Development Notes

Project Overview

Building a full-stack ticketing system with user authentication, ticket creation, assignment, and status tracking.

Tech Stack:

- **MongoDB** - Database for storing users, tickets, and related data
- **Express.js** - Backend framework and API routes
- **React** - Frontend user interface
- **Node.js** - Runtime environment

Project Structure

```
ticketing-app/
├── backend/
│   ├── config/
│   │   ├── db.js
│   │   └── auth.js
│   ├── controllers/
│   │   ├── authController.js
│   │   ├── ticketController.js
│   │   └── userController.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   └── errorMiddleware.js
│   ├── models/
│   │   ├── User.js
│   │   └── Ticket.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   ├── ticketRoutes.js
│   │   └── userRoutes.js
│   ├── utils/
│   │   └── generateToken.js
│   ├── .env
│   └── server.js
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
```

```
│   │   │   ├── auth/
│   │   │   ├── tickets/
│   │   │   ├── layout/
│   │   │   └── shared/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── context/
│   │   ├── services/
│   │   └── utils/
│   └── package.json
└── package.json
```

Backend Development

1. Initial Setup

Install Dependencies:

```bash
# Backend dependencies
npm init -y
npm install express mongoose dotenv bcryptjs jsonwebtoken cors helmet morgan
npm install -D nodemon concurrently

# Frontend dependencies (in frontend folder)
npx create-react-app frontend
cd frontend
npm install axios react-router-dom react-query @heroicons/react
```

Environment Variables (.env):

```
NODE_ENV=development
PORT=5000
MONGO_URI=mongodb://localhost:27017/ticketing-app
JWT_SECRET=your-super-secret-jwt-key
JWT_EXPIRE=7d
```

2. Database Configuration

config/db.js:

```javascript
```

```javascript
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

3. Database Models

models/User.js:

```javascript
```

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please add a name'],
  },
  email: {
    type: String,
    required: [true, 'Please add an email'],
    unique: true,
  },
  password: {
    type: String,
    required: [true, 'Please add a password'],
  },
  role: {
    type: String,
    enum: ['user', 'admin', 'support'],
    default: 'user',
  },
  isActive: {
    type: Boolean,
    default: true,
  },
}, {
  timestamps: true,
});

// Encrypt password before saving
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) {
    next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// Match user password
userSchema.methods.matchPassword = async function(enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

models/Ticket.js:

```javascript
```

```
const mongoose = require('mongoose');

const ticketSchema = mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Please add a title'],
  },
  description: {
    type: String,
    required: [true, 'Please add a description'],
  },
  status: {
    type: String,
    enum: ['open', 'in-progress', 'closed'],
    default: 'open',
  },
  priority: {
    type: String,
    enum: ['low', 'medium', 'high', 'urgent'],
    default: 'medium',
  },
  category: {
    type: String,
    enum: ['technical', 'billing', 'general', 'bug-report'],
    required: true,
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    required: true,
    ref: 'User',
  },
  assignedTo: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
  },
  comments: [{
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
    message: {
      type: String,
      required: true,
    },
    createdAt: {
```

```
      type: Date,
      default: Date.now,
    },
  }],
}, {
  timestamps: true,
});

module.exports = mongoose.model('Ticket', ticketSchema);
```

4. Authentication Utilities

utils/generateToken.js:

```javascript
const jwt = require('jsonwebtoken');

const generateToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRE,
  });
};

module.exports = generateToken;
```

middleware/authMiddleware.js:

```javascript
```

```javascript
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  let token;

  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      next();
    } catch (error) {
      res.status(401).json({ message: 'Not authorized, token failed' });
    }
  }

  if (!token) {
    res.status(401).json({ message: 'Not authorized, no token' });
  }
};

const admin = (req, res, next) => {
  if (req.user && (req.user.role === 'admin' || req.user.role === 'support')) {
    next();
  } else {
    res.status(403).json({ message: 'Access denied - Admin only' });
  }
};

module.exports = { protect, admin };
```

5. Controllers

controllers/authController.js:

```javascript
```

```
const User = require('../models/User');
const generateToken = require('../utils/generateToken');

// @desc Register new user
// @route POST /api/auth/register
const registerUser = async (req, res) => {
  try {
    const { name, email, password } = req.body;

    const userExists = await User.findOne({ email });
    if (userExists) {
      return res.status(400).json({ message: 'User already exists' });
    }

    const user = await User.create({ name, email, password });

    if (user) {
      res.status(201).json({
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        token: generateToken(user._id),
      });
    }
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

// @desc Auth user & get token
// @route POST /api/auth/login
const loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });

    if (user && (await user.matchPassword(password))) {
      res.json({
        _id: user._id,
        name: user.name,
        email: user.email,
        role: user.role,
        token: generateToken(user._id),
      });
```

```
    } else {
      res.status(401).json({ message: 'Invalid email or password' });
    }
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

// @desc Get user profile
// @route GET /api/auth/profile
const getUserProfile = async (req, res) => {
  try {
    const user = await User.findById(req.user._id).select('-password');
    res.json(user);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

module.exports = { registerUser, loginUser, getUserProfile };
```

controllers/ticketController.js:

```javascript
```

```
const Ticket = require('../models/Ticket');

// @desc Get all tickets
// @route GET /api/tickets
const getTickets = async (req, res) => {
  try {
    const query = req.user.role === 'user'
      ? { user: req.user._id }
      : {};

    const tickets = await Ticket.find(query)
      .populate('user', 'name email')
      .populate('assignedTo', 'name email')
      .sort({ createdAt: -1 });

    res.json(tickets);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

// @desc Create new ticket
// @route POST /api/tickets
const createTicket = async (req, res) => {
  try {
    const { title, description, priority, category } = req.body;

    const ticket = await Ticket.create({
      title,
      description,
      priority,
      category,
      user: req.user._id,
    });

    const populatedTicket = await Ticket.findById(ticket._id)
      .populate('user', 'name email');

    res.status(201).json(populatedTicket);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

// @desc Get single ticket
// @route GET /api/tickets/:id
```

```
const getTicket = async (req, res) => {
  try {
    const ticket = await Ticket.findById(req.params.id)
      .populate('user', 'name email')
      .populate('assignedTo', 'name email')
      .populate('comments.user', 'name email');

    if (!ticket) {
      return res.status(404).json({ message: 'Ticket not found' });
    }

    // Check ownership for regular users
    if (req.user.role === 'user' && ticket.user._id.toString() !== req.user._id.toString()) {
      return res.status(403).json({ message: 'Access denied' });
    }

    res.json(ticket);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

// @desc Update ticket
// @route PUT /api/tickets/:id
const updateTicket = async (req, res) => {
  try {
    const ticket = await Ticket.findById(req.params.id);

    if (!ticket) {
      return res.status(404).json({ message: 'Ticket not found' });
    }

    // Check permissions
    if (req.user.role === 'user' && ticket.user.toString() !== req.user._id.toString()) {
      return res.status(403).json({ message: 'Access denied' });
    }

    const updatedTicket = await Ticket.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    ).populate('user', 'name email').populate('assignedTo', 'name email');

    res.json(updatedTicket);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
```

```
};

// @desc Add comment to ticket
// @route POST /api/tickets/:id/comments
const addComment = async (req, res) => {
  try {
    const { message } = req.body;
    const ticket = await Ticket.findById(req.params.id);

    if (!ticket) {
      return res.status(404).json({ message: 'Ticket not found' });
    }

    const comment = {
      user: req.user._id,
      message,
    };

    ticket.comments.push(comment);
    await ticket.save();

    const updatedTicket = await Ticket.findById(req.params.id)
      .populate('comments.user', 'name email');

    res.json(updatedTicket);
  } catch (error) {
    res.status(400).json({ message: error.message });
  }
};

module.exports = {
  getTickets,
  createTicket,
  getTicket,
  updateTicket,
  addComment,
};
```

6. Routes

routes/authRoutes.js:

```javascript
```

```javascript
const express = require('express');
const { registerUser, loginUser, getUserProfile } = require('../controllers/authController');
const { protect } = require('../middleware/authMiddleware');

const router = express.Router();

router.post('/register', registerUser);
router.post('/login', loginUser);
router.get('/profile', protect, getUserProfile);

module.exports = router;
```

routes/ticketRoutes.js:

```javascript
const express = require('express');
const {
  getTickets,
  createTicket,
  getTicket,
  updateTicket,
  addComment,
} = require('../controllers/ticketController');
const { protect, admin } = require('../middleware/authMiddleware');

const router = express.Router();

router.route('/')
  .get(protect, getTickets)
  .post(protect, createTicket);

router.route('/:id')
  .get(protect, getTicket)
  .put(protect, updateTicket);

router.post('/:id/comments', protect, addComment);

module.exports = router;
```

7. Server Setup

server.js:

```javascript
```

```
const express = require('express');
const dotenv = require('dotenv');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');
const connectDB = require('./config/db');

// Load env vars
dotenv.config();

// Connect to database
connectDB();

const app = express();

// Middleware
app.use(helmet());
app.use(cors());
app.use(morgan('combined'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

// Routes
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/tickets', require('./routes/ticketRoutes'));

// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: 'Server Error' });
});

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Frontend Development

1. Project Structure Setup

Create folder structure in src/:

- components/ (reusable UI components)

- pages/ (main page components)
- context/ (React context for state management)
- services/ (API calls)
- hooks/ (custom React hooks)
- utils/ (helper functions)

2. Authentication Context

src/context/AuthContext.js:

```javascript
```

```
import { createContext, useContext, useReducer, useEffect } from 'react';
import axios from 'axios';

const AuthContext = createContext();

const authReducer = (state, action) => {
  switch (action.type) {
    case 'LOGIN_SUCCESS':
      localStorage.setItem('token', action.payload.token);
      return {
        ...state,
        user: action.payload,
        token: action.payload.token,
        isAuthenticated: true,
        loading: false,
      };
    case 'LOGOUT':
      localStorage.removeItem('token');
      return {
        ...state,
        user: null,
        token: null,
        isAuthenticated: false,
        loading: false,
      };
    case 'SET_LOADING':
      return {
        ...state,
        loading: action.payload,
      };
    default:
      return state;
  }
};

export const AuthProvider = ({ children }) => {
  const [state, dispatch] = useReducer(authReducer, {
    user: null,
    token: localStorage.getItem('token'),
    isAuthenticated: false,
    loading: true,
  });

  useEffect(() => {
    const token = localStorage.getItem('token');
    if (token) {
```

```
    axios.defaults.headers.common['Authorization'] = `Bearer ${token}`;
    // Verify token and get user data
    loadUser();
  } else {
    dispatch({ type: 'SET_LOADING', payload: false });
  }
}, []);

const loadUser = async () => {
  try {
    const res = await axios.get('/api/auth/profile');
    dispatch({
      type: 'LOGIN_SUCCESS',
      payload: { ...res.data, token: state.token },
    });
  } catch (error) {
    dispatch({ type: 'LOGOUT' });
  }
};

const login = async (email, password) => {
  try {
    const res = await axios.post('/api/auth/login', { email, password });
    dispatch({ type: 'LOGIN_SUCCESS', payload: res.data });
    axios.defaults.headers.common['Authorization'] = `Bearer ${res.data.token}`;
  } catch (error) {
    throw error;
  }
};

const register = async (name, email, password) => {
  try {
    const res = await axios.post('/api/auth/register', { name, email, password });
    dispatch({ type: 'LOGIN_SUCCESS', payload: res.data });
    axios.defaults.headers.common['Authorization'] = `Bearer ${res.data.token}`;
  } catch (error) {
    throw error;
  }
};

const logout = () => {
  dispatch({ type: 'LOGOUT' });
  delete axios.defaults.headers.common['Authorization'];
};

return (
  <AuthContext.Provider value={{
```

```
      ...state,
      login,
      register,
      logout,
    }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
};
```

3. API Service

src/services/api.js:

```javascript
import axios from 'axios';

const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/api';

axios.defaults.baseURL = API_URL;

// Tickets API
export const ticketsAPI = {
  getAll: () => axios.get('/tickets'),
  getById: (id) => axios.get(`/tickets/${id}`),
  create: (ticketData) => axios.post('/tickets', ticketData),
  update: (id, ticketData) => axios.put(`/tickets/${id}`, ticketData),
  addComment: (id, message) => axios.post(`/tickets/${id}/comments`, { message }),
};

// Auth API
export const authAPI = {
  login: (credentials) => axios.post('/auth/login', credentials),
  register: (userData) => axios.post('/auth/register', userData),
  getProfile: () => axios.get('/auth/profile'),
};
```

4. Key Components

src/components/auth/LoginForm.js:

```javascript
```

```
import { useState } from 'react';
import { useAuth } from '../../context/AuthContext';
import { useNavigate } from 'react-router-dom';

const LoginForm = () => {
  const [formData, setFormData] = useState({ email: '', password: '' });
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError('');

    try {
      await login(formData.email, formData.password);
      navigate('/dashboard');
    } catch (error) {
      setError(error.response?.data?.message || 'Login failed');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="max-w-md mx-auto bg-white rounded-lg shadow-md p-6">
      <h2 className="text-2xl font-bold mb-6">Login</h2>

      {error && (
        <div className="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded mb-4">
          {error}
        </div>
      )}

      <form onSubmit={handleSubmit}>
        <div className="mb-4">
          <label className="block text-gray-700 text-sm font-bold mb-2">
            Email
          </label>
          <input
            type="email"
            value={formData.email}
            onChange={(e) => setFormData({ ...formData, email: e.target.value })}
            className="w-full px-3 py-2 border rounded-lg focus:outline-none focus:border-blue-500"
```

```
          required
        />
      </div>

      <div className="mb-6">
        <label className="block text-gray-700 text-sm font-bold mb-2">
          Password
        </label>
        <input
          type="password"
          value={formData.password}
          onChange={(e) => setFormData({ ...formData, password: e.target.value })}
          className="w-full px-3 py-2 border rounded-lg focus:outline-none focus:border-blue-500"
          required
        />
      </div>

      <button
        type="submit"
        disabled={loading}
        className="w-full bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded focus:outline-none f
      >
        {loading ? 'Logging in...' : 'Login'}
      </button>
    </form>
  </div>
  );
};

export default LoginForm;
```

src/components/tickets/TicketForm.js:

```javascript
```

```
import { useState } from 'react';
import { ticketsAPI } from '../../services/api';

const TicketForm = ({ onSuccess, ticket = null }) => {
  const [formData, setFormData] = useState({
    title: ticket?.title || '',
    description: ticket?.description || '',
    priority: ticket?.priority || 'medium',
    category: ticket?.category || 'general',
  });
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    setError('');

    try {
      if (ticket) {
        await ticketsAPI.update(ticket._id, formData);
      } else {
        await ticketsAPI.create(formData);
      }
      onSuccess && onSuccess();
    } catch (error) {
      setError(error.response?.data?.message || 'Failed to save ticket');
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-4">
      {error && (
        <div className="bg-red-100 border border-red-400 text-red-700 px-4 py-3 rounded">
          {error}
        </div>
      )}

      <div>
        <label className="block text-sm font-medium text-gray-700">Title</label>
        <input
          type="text"
          value={formData.title}
          onChange={(e) => setFormData({ ...formData, title: e.target.value })}
```

```
    className="mt-1 block w-full border-gray-300 rounded-md shadow-sm focus:border-indigo-500 focus:ring-ind
    required
  />
</div>

<div>
  <label className="block text-sm font-medium text-gray-700">Description</label>
  <textarea
    value={formData.description}
    onChange={(e) => setFormData({ ...formData, description: e.target.value })}
    rows={4}
    className="mt-1 block w-full border-gray-300 rounded-md shadow-sm focus:border-indigo-500 focus:ring-ind
    required
  />
</div>

<div className="grid grid-cols-2 gap-4">
  <div>
    <label className="block text-sm font-medium text-gray-700">Priority</label>
    <select
      value={formData.priority}
      onChange={(e) => setFormData({ ...formData, priority: e.target.value })}
      className="mt-1 block w-full border-gray-300 rounded-md shadow-sm focus:border-indigo-500 focus:ring-in
    >
      <option value="low">Low</option>
      <option value="medium">Medium</option>
      <option value="high">High</option>
      <option value="urgent">Urgent</option>
    </select>
  </div>

  <div>
    <label className="block text-sm font-medium text-gray-700">Category</label>
    <select
      value={formData.category}
      onChange={(e) => setFormData({ ...formData, category: e.target.value })}
      className="mt-1 block w-full border-gray-300 rounded-md shadow-sm focus:border-indigo-500 focus:ring-in
    >
      <option value="general">General</option>
      <option value="technical">Technical</option>
      <option value="billing">Billing</option>
      <option value="bug-report">Bug Report</option>
    </select>
  </div>
</div>

<button
```

```
      type="submit"
      disabled={loading}
      className="w-full flex justify-center py-2 px-4 border border-transparent rounded-md shadow-sm text-sm font-
    >
      {loading ? 'Saving...' : (ticket ? 'Update Ticket' : 'Create Ticket')}
    </button>
  </form>
 );
};

export default TicketForm;
```

5. Main Pages

src/pages/Dashboard.js:

```javascript
```

```
import { useState, useEffect } from 'react';
import { ticketsAPI } from '../services/api';
import { useAuth } from '../context/AuthContext';
import TicketList from '../components/tickets/TicketList';
import TicketForm from '../components/tickets/TicketForm';

const Dashboard = () => {
  const [tickets, setTickets] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showForm, setShowForm] = useState(false);
  const { user } = useAuth();

  const fetchTickets = async () => {
    try {
      const response = await ticketsAPI.getAll();
      setTickets(response.data);
    } catch (error) {
      console.error('Error fetching tickets:', error);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchTickets();
  }, []);

  const handleTicketSuccess = () => {
    setShowForm(false);
    fetchTickets();
  };

  if (loading) {
    return <div className="flex justify-center items-center h-64">Loading...</div>;
  }

  return (
    <div className="container mx-auto px-4 py-8">
      <div className="flex justify-between items-center mb-6">
        <h1 className="text-3xl font-bold text-gray-900">
          Welcome, {user?.name}
        </h1>
        <button
          onClick={() => setShowForm(!showForm)}
          className="bg-blue-500 hover:bg-blue-700 text-white font-bold py-2 px-4 rounded"
        >
```

```
        {showForm ? 'Cancel' : 'New Ticket'}
      </button>
    </div>

    {showForm && (
      <div className="mb-8 bg-white p-6 rounded-lg shadow">
        <h2 className="text-xl font-semibold mb-4">Create New Ticket</h2>
        <TicketForm onSuccess={handleTicketSuccess} />
      </div>
    )}

    <div className="bg-white rounded-lg shadow">
      <div className="px-6 py-4 border-b">
        <h2 className="text-xl font-semibold">Your Tickets</h2>
      </div>
      <TicketList tickets={tickets} onUpdate={fetchTickets} />
    </div>
  </div>
  );
};

export default Dashboard;
```

6. App.js Setup

src/App.js:

```javascript
```

```
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './context/AuthContext';
import Navbar from './components/layout/Navbar';
import Dashboard from './pages/Dashboard';
import Login from './pages/Login';
import Register from './pages/Register';
import TicketDetail from './pages/TicketDetail';

const ProtectedRoute = ({ children }) => {
  const { isAuthenticated, loading } = useAuth();

  if (loading) {
    return <div className="flex justify-center items-center h-screen">Loading...</div>;
  }

  return isAuthenticated ? children : <Navigate to="/login" />;
};

function App() {
  return (
    <AuthProvider>
      <Router>
        <div className="min-h-screen bg-gray-100">
          <Navbar />
          <main>
            <Routes>
              <Route path="/login" element={<Login />} />
              <Route path="/register" element={<Register />} />
              <Route path="/" element={<Navigate to="/dashboard" />} />
              <Route
                path="/dashboard"
                element={
                  <ProtectedRoute>
                    <Dashboard />
                  </ProtectedRoute>
                }
              />
              <Route
                path="/tickets/:id"
                element={
                  <ProtectedRoute>
                    <TicketDetail />
                  </ProtectedRoute>
                }
              />
            </Routes>
```

```
        </main>
      </div>
    </Router>
  </AuthProvider>
 );
}

export default App;
```

Additional Components & Features

7. Navigation Component

src/components/layout/Navbar.js:

```javascript
```

```
import { useAuth } from '../../context/AuthContext';
import { Link } from 'react-router-dom';

const Navbar = () => {
  const { isAuthenticated, user, logout } = useAuth();

  const handleLogout = () => {
    logout();
  };

  return (
    <nav className="bg-blue-600 shadow-lg">
      <div className="max-w-7xl mx-auto px-4">
        <div className="flex justify-between items-center h-16">
          <Link to="/" className="text-white text-xl font-bold">
            TicketHub
          </Link>

          {isAuthenticated ? (
            <div className="flex items-center space-x-4">
              <span className="text-white">Welcome, {user?.name}</span>
              <span className="bg-blue-700 text-white px-2 py-1 rounded-full text-xs">
                {user?.role}
              </span>
              <Link
                to="/dashboard"
                className="text-white hover:text-blue-200"
              >
                Dashboard
              </Link>
              <button
                onClick={handleLogout}
                className="bg-blue-700 hover:bg-blue-800 text-white px-4 py-2 rounded"
              >
                Logout
              </button>
            </div>
          ) : (
            <div className="space-x-4">
              <Link
                to="/login"
                className="text-white hover:text-blue-200"
              >
                Login
              </Link>
              <Link
```

```
            to="/register"
            className="bg-blue-700 hover:bg-blue-800 text-white px-4 py-2 rounded"
          >
            Sign Up
          </Link>
        </div>
      )}
    </div>
  </div>
</nav>
);
};

export default Navbar;
```

8. Ticket List Component

src/components/tickets/TicketList.js:

```javascript
```

```
import { Link } from 'react-router-dom';

const TicketList = ({ tickets, onUpdate }) => {
  const getPriorityColor = (priority) => {
    const colors = {
      low: 'bg-green-100 text-green-800',
      medium: 'bg-yellow-100 text-yellow-800',
      high: 'bg-orange-100 text-orange-800',
      urgent: 'bg-red-100 text-red-800',
    };
    return colors[priority] || 'bg-gray-100 text-gray-800';
  };

  const getStatusColor = (status) => {
    const colors = {
      open: 'bg-blue-100 text-blue-800',
      'in-progress': 'bg-purple-100 text-purple-800',
      closed: 'bg-gray-100 text-gray-800',
    };
    return colors[status] || 'bg-gray-100 text-gray-800';
  };

  if (tickets.length === 0) {
    return (
      <div className="p-6 text-center text-gray-500">
        No tickets found. Create your first ticket!
      </div>
    );
  }

  return (
    <div className="overflow-x-auto">
      <table className="min-w-full divide-y divide-gray-200">
        <thead className="bg-gray-50">
          <tr>
            <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
              Title
            </th>
            <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
              Status
            </th>
            <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
              Priority
            </th>
            <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
              Category
```

```
    </th>
    <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
      Created
    </th>
    <th className="px-6 py-3 text-left text-xs font-medium text-gray-500 uppercase tracking-wider">
      Actions
    </th>
  </tr>
</thead>
<tbody className="bg-white divide-y divide-gray-200">
 {tickets.map((ticket) => (
   <tr key={ticket._id} className="hover:bg-gray-50">
     <td className="px-6 py-4 whitespace-nowrap">
       <div className="text-sm font-medium text-gray-900">
         {ticket.title}
       </div>
       <div className="text-sm text-gray-500 truncate max-w-xs">
         {ticket.description}
       </div>
     </td>
     <td className="px-6 py-4 whitespace-nowrap">
       <span className={`inline-flex px-2 py-1 text-xs font-semibold rounded-full ${getStatusColor(ticket.status)}`}>
         {ticket.status}
       </span>
     </td>
     <td className="px-6 py-4 whitespace-nowrap">
       <span className={`inline-flex px-2 py-1 text-xs font-semibold rounded-full ${getPriorityColor(ticket.priority)}
         {ticket.priority}
       </span>
     </td>
     <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-900">
      {ticket.category}
     </td>
     <td className="px-6 py-4 whitespace-nowrap text-sm text-gray-500">
      {new Date(ticket.createdAt).toLocaleDateString()}
     </td>
     <td className="px-6 py-4 whitespace-nowrap text-sm font-medium">
       <Link
         to={`/tickets/${ticket._id}`}
         className="text-indigo-600 hover:text-indigo-900"
       >
         View Details
       </Link>
     </td>
   </tr>
 ))}
</tbody>
```

```
      </table>
    </div>
  );
};

export default TicketList;
```

9. Ticket Detail Page

src/pages/TicketDetail.js:

```javascript
```

```
import { useState, useEffect } from 'react';
import { useParams } from 'react-router-dom';
import { ticketsAPI } from '../services/api';
import { useAuth } from '../context/AuthContext';
import CommentSection from '../components/tickets/CommentSection';
import TicketForm from '../components/tickets/TicketForm';

const TicketDetail = () => {
  const { id } = useParams();
  const [ticket, setTicket] = useState(null);
  const [loading, setLoading] = useState(true);
  const [editing, setEditing] = useState(false);
  const [error, setError] = useState('');
  const { user } = useAuth();

  const fetchTicket = async () => {
    try {
      const response = await ticketsAPI.getById(id);
      setTicket(response.data);
    } catch (error) {
      setError('Failed to load ticket');
      console.error('Error fetching ticket:', error);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchTicket();
  }, [id]);

  const handleUpdateSuccess = () => {
    setEditing(false);
    fetchTicket();
  };

  const handleStatusUpdate = async (newStatus) => {
    try {
      await ticketsAPI.update(id, { status: newStatus });
      fetchTicket();
    } catch (error) {
      console.error('Error updating status:', error);
    }
  };

  if (loading) {
```

```
  return <div className="flex justify-center items-center h-64">Loading...</div>;
}

if (error) {
  return <div className="text-center text-red-600 p-4">{error}</div>;
}

if (!ticket) {
  return <div className="text-center p-4">Ticket not found</div>;
}

const canEdit = user?.role === 'admin' || user?.role === 'support' ||
              (user?._id === ticket.user._id && ticket.status === 'open');

return (
  <div className="container mx-auto px-4 py-8">
    <div className="max-w-4xl mx-auto">
      {/* Header */}
      <div className="bg-white rounded-lg shadow-md p-6 mb-6">
        <div className="flex justify-between items-start mb-4">
          <h1 className="text-2xl font-bold text-gray-900">{ticket.title}</h1>
          <div className="flex space-x-2">
            {canEdit && (
              <button
                onClick={() => setEditing(!editing)}
                className="bg-blue-500 hover:bg-blue-700 text-white px-4 py-2 rounded"
              >
                {editing ? 'Cancel Edit' : 'Edit'}
              </button>
            )}
            {(user?.role === 'admin' || user?.role === 'support') && (
              <div className="space-x-2">
                <select
                  value={ticket.status}
                  onChange={(e) => handleStatusUpdate(e.target.value)}
                  className="border rounded px-3 py-2"
                >
                  <option value="open">Open</option>
                  <option value="in-progress">In Progress</option>
                  <option value="closed">Closed</option>
                </select>
              </div>
            )}
          </div>
        </div>

        {editing ? (
```

```
    <TicketForm ticket={ticket} onSuccess={handleUpdateSuccess} />
  ) : (
    <>
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4 mb-4">
        <div>
          <span className="text-sm text-gray-500">Status</span>
          <p className="font-medium">{ticket.status}</p>
        </div>
        <div>
          <span className="text-sm text-gray-500">Priority</span>
          <p className="font-medium">{ticket.priority}</p>
        </div>
        <div>
          <span className="text-sm text-gray-500">Category</span>
          <p className="font-medium">{ticket.category}</p>
        </div>
        <div>
          <span className="text-sm text-gray-500">Created</span>
          <p className="font-medium">
            {new Date(ticket.createdAt).toLocaleDateString()}
          </p>
        </div>
      </div>

      <div className="mb-4">
        <span className="text-sm text-gray-500">Description</span>
        <p className="mt-1 text-gray-900 whitespace-pre-wrap">
          {ticket.description}
        </p>
      </div>

      <div className="flex justify-between text-sm text-gray-500">
        <span>Created by: {ticket.user.name}</span>
        {ticket.assignedTo && (
          <span>Assigned to: {ticket.assignedTo.name}</span>
        )}
      </div>
    </>
  )}
</div>

{/* Comments Section */}
<CommentSection
  ticketId={ticket._id}
  comments={ticket.comments}
  onUpdate={fetchTicket}
/>
```

```
      </div>
    </div>
  );
};

export default TicketDetail;
```

10. Comment Component

src/components/tickets/CommentSection.js:

```javascript
```

```
import { useState } from 'react';
import { ticketsAPI } from '../../services/api';
import { useAuth } from '../../context/AuthContext';

const CommentSection = ({ ticketId, comments, onUpdate }) => {
  const [newComment, setNewComment] = useState('');
  const [loading, setLoading] = useState(false);
  const { user } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!newComment.trim()) return;

    setLoading(true);
    try {
      await ticketsAPI.addComment(ticketId, newComment);
      setNewComment('');
      onUpdate();
    } catch (error) {
      console.error('Error adding comment:', error);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="bg-white rounded-lg shadow-md p-6">
      <h3 className="text-lg font-semibold mb-4">
        Comments ({comments.length})
      </h3>

      {/* Comment Form */}
      <form onSubmit={handleSubmit} className="mb-6">
        <textarea
          value={newComment}
          onChange={(e) => setNewComment(e.target.value)}
          placeholder="Add a comment..."
          className="w-full p-3 border rounded-lg resize-none focus:outline-none focus:border-blue-500"
          rows="3"
        />
        <button
          type="submit"
          disabled={loading || !newComment.trim()}
          className="mt-2 bg-blue-500 hover:bg-blue-700 text-white px-4 py-2 rounded disabled:opacity-50"
        >
          {loading ? 'Adding...' : 'Add Comment'}
```

```
      </button>
    </form>

    {/* Comments List */}
    <div className="space-y-4">
      {comments.map((comment, index) => (
        <div key={index} className="border-l-4 border-gray-200 pl-4">
          <div className="flex justify-between items-start mb-1">
            <span className="font-medium text-gray-900">
              {comment.user.name}
            </span>
            <span className="text-sm text-gray-500">
              {new Date(comment.createdAt).toLocaleString()}
            </span>
          </div>
          <p className="text-gray-700 whitespace-pre-wrap">
            {comment.message}
          </p>
        </div>
      ))}

      {comments.length === 0 && (
        <p className="text-gray-500 text-center py-4">
          No comments yet. Be the first to comment!
        </p>
      )}
    </div>
  </div>
  );
};

export default CommentSection;
```

Package.json Scripts

Backend package.json:

```
json
```

```
{
  "name": "ticketing-backend",
  "version": "1.0.0",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "client": "npm start --prefix ../frontend",
    "devall": "concurrently \"npm run dev\" \"npm run client\""
  }
}
```

Frontend package.json:

```json
{
  "name": "ticketing-frontend",
  "version": "1.0.0",
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
    "eject": "react-scripts eject"
  },
  "proxy": "http://localhost:5000"
}
```

Development Workflow

1. Initial Setup Steps:

```bash
```

```bash
# Create project directory
mkdir ticketing-app && cd ticketing-app

# Backend setup
mkdir backend && cd backend
npm init -y
# Install dependencies (see above)
# Create folder structure and files

# Frontend setup
cd ..
npx create-react-app frontend
cd frontend
# Install additional dependencies
# Create folder structure and components
```

2. Running the Application:

```bash
# Terminal 1 - Backend
cd backend
npm run dev

# Terminal 2 - Frontend
cd frontend
npm start

# Or run both simultaneously from backend:
npm run devall
```

3. Environment Setup:

- MongoDB running locally or MongoDB Atlas connection
- Create .env file with proper environment variables
- Set up Tailwind CSS for styling (optional)

Key Features Implemented

✅ **Authentication System:**

- User registration and login
- JWT token-based authentication

- Protected routes
- Role-based access control

✅ **Ticket Management:**

- Create, read, update tickets
- Priority and category classification
- Status tracking (open, in-progress, closed)
- User ownership and assignment

✅ **Comment System:**

- Add comments to tickets
- Real-time comment updates
- User attribution

✅ **User Interface:**

- Responsive design with Tailwind CSS
- Dashboard with ticket overview
- Detailed ticket view
- Form validation and error handling

✅ **Security Features:**

- Password hashing with bcrypt
- JWT authentication
- Input validation
- CORS protection

Next Steps & Enhancements

Immediate Improvements:

1. **File Attachments** - Allow users to attach files to tickets
2. **Email Notifications** - Send emails for ticket updates
3. **Search & Filtering** - Advanced ticket search capabilities
4. **Real-time Updates** - WebSocket integration for live updates
5. **Admin Panel** - User management and system analytics

Advanced Features:

1. **Ticket Templates** - Pre-defined ticket templates
2. **SLA Management** - Service level agreement tracking
3. **Knowledge Base** - FAQ and documentation system
4. **Reporting & Analytics** - Dashboard with charts and metrics
5. **Mobile App** - React Native mobile application

Production Considerations:

1. **Testing** - Unit tests with Jest, integration tests
2. **Deployment** - Docker containerization, CI/CD pipeline
3. **Monitoring** - Error tracking, performance monitoring
4. **Scaling** - Database indexing, caching strategies
5. **Security** - Rate limiting, input sanitization, security headers

---

Troubleshooting Common Issues

Backend Issues:

- **MongoDB Connection**: Check connection string and database status
- **JWT Errors**: Verify JWT_SECRET is set and tokens are properly formatted
- **CORS Issues**: Ensure CORS is configured for frontend URL

Frontend Issues:

- **Proxy Issues**: Check proxy setting in package.json points to backend
- **Auth State**: Verify token storage and context provider setup
- **API Calls**: Check network tab for failed requests and error responses

General Development Tips:

- Use MongoDB Compass for database visualization
- Install React Developer Tools for debugging
- Use Postman for API testing
- Check browser console for JavaScript errors
- Monitor network requests for API issues